



ECA304

PROGRAMAÇÃO

ORIENTADA A OBJETOS

Alexandre H. MOREIRA

CLASSES E OBJETOS — MÉTODOS DUNDER



O que são Dunder Methods?

Métodos Especiais em Python, também conhecidos como **Dunder Methods** (de *Double UNDERscore*)

- São métodos com **dois underlines antes e depois do nome**.
- Permitem **personalizar o comportamento interno dos objetos**.
- Chamados **automaticamente** pelo Python em determinadas situações:
 - **__init__**: chamado quando o objeto é criado;
 - **__str__**: chamado quando usamos **print(objeto)**;
 - **__len__**: chamado quando usamos **len(objeto)**;



Principais Dunder Methods

Classes e Objetos - Dunder

Método	Quando é chamado	Exemplo
<code>__init__</code>	Ao instanciar um objeto	<code>obj = Pessoa()</code>
<code>__str__</code>	Ao usar <code>print(objeto)</code>	<code>print(obj)</code>
<code>__repr__</code>	Ao usar o objeto no terminal ou <code>repr(objeto)</code>	<code>repr(obj)</code>
<code>__eq__</code>	Ao comparar objetos com <code>==</code>	<code>obj1 == obj2</code>
<code>__lt__</code>	Ao comparar objetos com <code><</code>	<code>obj1 < obj2</code>
<code>__len__</code>	Ao usar <code>len(objeto)</code>	<code>len(obj)</code>
<code>__getitem__</code>	Ao acessar como índice	<code>obj[0]</code>
<code>__setitem__</code>	Ao atribuir um valor a um índice	<code>obj[0] = valor</code>
<code>__contains__</code>	Ao usar <code>in</code>	<code>"chave" in obj</code>



Dunder Methods

Exemplo

```
class CarrinhoDeCompras:
    def __init__(self):
        self.itens = []

    def adicionar(self, item):
        self.itens.append(item)

    def __len__(self):
        return len(self.itens)

    def __getitem__(self, index):
        return self.itens[index]

    def __contains__(self, item):
        return item in self.itens
```



Dunder Methods

Exemplo

```
print(a1)           # Usa __str__  
print(a1 == a3)     # True, mesmo RA  
print(a1 < a2)      # True, menor nota
```

```
Aluno: Alice - Nota: 8.5  
True  
True
```

Diferença entre `__str__` e `__repr__`

```
print(a1)           # Usa __str__  
print(repr(a1))     # Usa __repr__
```

```
Aluno: Alice - Nota: 8.5  
Aluno('Alice', 101, 8.5)
```



Diferença entre `__str__` e `__repr__`

Classes e Objetos - Dunder

O método `__repr__` é utilizado principalmente quando se trata de **debugar, testar e representar objetos de forma precisa**.

“O `__repr__` de um objeto deve, se possível, ser uma string válida de Python que poderia ser usada para recriar o objeto” — *Guia de estilo do Python (PEP 8)*



Dunder Methods

Exemplo

```
class CarrinhoDeCompras:
    def __init__(self):
        self.itens = []

    def adicionar(self, item):
        self.itens.append(item)

    def __len__(self):
        return len(self.itens)

    def __getitem__(self, index):
        return self.itens[index]

    def __contains__(self, item):
        return item in self.itens
```

Dunder Methods - Operadores Numéricos



Classes e Objetos - Dunder

Operador	Expressão	Método
Addition	$q1 + q2$	<code>__add__(self, q2)</code>
Subtraction	$q1 - q2$	<code>__sub__(self, q2)</code>
Multiplication	$q1 * q2$	<code>__mul__(self, q2)</code>
Power	$q1 ** q2$	<code>__pow__(self, q2)</code>
Division	$q1 / q2$	<code>__truediv__(self, q2)</code>
Floor Division	$q1 // q2$	<code>__floordiv__(self, q2)</code>
Modulo (Remainder)	$q1 \% q2$	<code>__mod__(self, q2)</code>
Bitwise Left Shift	$q1 \ll q2$	<code>__lshift__(self, q2)</code>
Bitwise Right Shift	$q1 \gg q2$	<code>__rshift__(self, q2)</code>



Dunder Methods - Operadores de Comparação

Classes e Objetos - Dunder

Operador	Expressão	Método
Less than	$q1 < q2$	<code>__lt__(q1, q2)</code>
Less than or equal to	$q1 \leq q2$	<code>__le__(q1, q2)</code>
Equal to	$q1 == q2$	<code>__eq__(q1, q2)</code>
Not Equal to	$q1 \neq q2$	<code>__ne__(q1, q2)</code>
Greater than	$q1 > q2$	<code>__gt__(q1, q2)</code>
Greater than or equal to	$q1 \geq q2$	<code>__ge__(q1, q2)</code>

Dunder Methods - Operadores Lógicos



Classes e Objetos - Dunder

Operador	Expressão	Método
AND	$q1 \& q2$	<code>__and__(q1, q2)</code>
OR	$q1 q2$	<code>__or__(q1, q2)</code>
XOR	$q1 \wedge q2$	<code>__xor__(q1, q2)</code>
NOT	$\sim q1$	<code>__invert__()</code>



EXERCÍCIO PARA ENTREGA

Sua vez!



Exercício para entrega

Classes e Objetos - Dunder



Cenário:

Modela um sistema de biblioteca com:

- Uma classe **livro** (objeto individual);
- Uma classe **biblioteca** (coleção de livros);
- Uso de métodos dunder.



Exercício para entrega

- **Classe livro:**

- Atributos: título, autor, ano, páginas;
- Métodos especiais:
 - **__str__**: “Título (ano) – Autor”
 - **__repr__**: “livro(‘Título’, ‘Autor’, Ano, Páginas)”
 - **__eq__**: Dois livros são iguais se título e autor forem iguais
 - **__lt__**: Menor livro = menos páginas



Exercício para entrega

- **Classe biblioteca:**
 - Atributos: livros (lista de objetos **livro**);
 - Métodos:
 - adicionar_livro(livro)
 - remover_livro(livro)
 - Métodos especiais:
 - **__len__**: número de livros
 - **__contains__**: verifica se um livro está na biblioteca
 - **__getitem__**: acessa um livro pelo índice
 - **__str__**: lista os livros com índice e título



Exercício para entrega

□ Exemplo de uso esperado:

```
livro1 = Livro("1984", "George Orwell", 1949, 328)
livro2 = Livro("Admirável Mundo Novo", "Aldous Huxley", 1932, 288)

biblioteca = Biblioteca()
biblioteca.adicionar_livro(livro1)
biblioteca.adicionar_livro(livro2)

print(biblioteca)           # Lista todos os livros
print(len(biblioteca))      # Deve retornar 2
print(livro1 in biblioteca) # True
print(biblioteca[0])        # Acessa livro pelo índice
```